We are IntechOpen, the world's leading publisher of Open Access books Built by scientists, for scientists

7,100
Open access books available

189,000
International authors and editors

205M
Downloads

Our authors are among the

154
Countries delivered to

TOP 1%
most cited scientists

12.2%
Contributors from top 500 universities



WEB OF SCIENCE™

Selection of our books indexed in the Book Citation Index in Web of Science™ Core Collection (BKCI)

Interested in publishing with us?
Contact book.department@intechopen.com

Numbers displayed above are based on latest data collected.
For more information visit www.intechopen.com



Chapter

Physics-Informed Neural Network: Principles and Applications

Seho Son, Jinho Jeong, Dayeon Jeong, Kyung ho Sun and Ki-Yong Oh

Abstract

This chapter delves into the fascinating characteristics of physics-informed neural networks (PINNs) by outlining their fundamental principles, including their mathematical foundations and structures. PINNs are designed by incorporating governing physical equations into the loss function as constraints, which helps to ensure precise output predictions even in areas with limited or no data. This chapter presents various strategies to apply PINNs to complex systems, thereby addressing the shortcomings of conventional PINNs. Additionally, multiphysics-informed neural networks (MPINNs) are introduced, with a special emphasis on complex mechatronic systems. The effectiveness of the MPINN framework is illustrated through examples such as an electric motor and a lithium-ion battery, demonstrating accurate and efficient multidimensional predictions for mechatronic systems despite limited data availability. These applications underscore the potential of MPINNs to mitigate data scarcity challenges in various industries.

Keywords: deep neural network, fundamental principles, governing equation, mechatronic system, physics-informed neural network, prognosis and health management, supervision of multiphysics

1. Introduction

Deep neural networks (DNNs) are widely used in various domains, including natural language processing [1–3] and image recognition [4–8]. For natural language processing, DNNs are employed in tasks like machine translation, sentiment analysis, and speech recognition, significantly enhancing the understanding and generation of human language. In the field of image recognition, DNNs are instrumental in object classification, feature detection, and image segmentation, finding applications in facial recognition systems and medical imaging diagnostics. The application of DNNs has surged in recent years across various scientific and engineering fields [9–13], including mechatronics, fluid dynamics, materials science, and biomedical engineering. The universal approximation theorem highlights the capability of DNNs to model any continuous function, enabling the effective estimation of complex and nonlinear behaviors in diverse systems [14]. Specifically, DNNs capture complex and nonlinear relationships between input and output through a training process that optimizes a

loss function quantifying the discrepancy between predicted and actual output. Therefore, the performance of DNNs significantly depends on the volume of data available to account for system characteristics, suggesting that collecting sufficient high-quality data is crucial for ensuring robustness and generality in real-world applications. This extensive data requirement can be costly and resource-intensive, constraining the accurate prediction of system behaviors when relying on purely data-driven approaches [15]. Furthermore, DNNs are often considered "black boxes" because the decision-making processes are not easily interpretable. This lack of transparency in data-only training can be problematic in critical applications, where understanding the model's inferences is essential for trust and validation.

To overcome these limitations, research has shifted toward hybrid approaches that combine the robustness of physics-based models with the adaptability of data-driven methods [16–18]. One notable strategy in this area involves incorporating knowledge-based features derived through feature engineering. This approach addresses domain-specific knowledge to enhance model accuracy by integrating the strengths of both fundamental physics and data-driven predictions. However, these methods still struggle with managing noisy data and outliers effectively [19]. Data noise can obscure underlying relationships between input and output, complicating the learning process for models. Outliers, on the other hand, can distort model predictions, leading to unreliable results. Meanwhile, physics-informed neural networks (PINNs) have gained attention as a powerful hybrid approach in that this method could enhance the fidelity of model predictions by incorporating physics-based regularization directly into the learning process. Specifically, PINNs minimize a loss function that combines both data-derived and physical laws to ensure robust predictions even under data scarcity or noisy situation. These networks are versatile in handling both forward and inverse problems across various time and space scales [20]. For example, Raissi et al. [21] demonstrated the application of PINNs in solving forward and inverse problems for nonlinear PDEs, showcasing their potential in accurately modeling physical systems with limited data. Another study by Tartakovsky et al. [22] applied PINNs to subsurface flow problems, achieving high accuracy in predicting hydraulic conductivity fields.

Despite these advantages, PINNs face challenges such as addressing stiff partial differential equations (PDEs) and managing multi-physics in complex systems. Issues such as spectral bias, where neural networks struggle to accurately capture high-frequency components of solutions [23], and difficulties in applying PINNs to parametric PDEs, which involve varying parameters that change the behavior of the system [24], have further been identified. These challenges can impact the accuracy and robustness of PINNs, necessitating further research and development to enhance their performance in diverse applications. Therefore, various methods have been actively suggested to overcome these challenges, including adaptively weighting each loss term [25–28], domain decomposition [29, 30], improved architecture to manage interpolation and extrapolation [31, 32], and embedding input coordinate features [32–34]. These efforts extend the application of PINNs beyond solid mechanics [35], thermal dynamics [36, 37] fluid mechanics [38, 39], and electromagnetics [40], enabling practical use in multiphysics system. Furthermore, increasing insight of overcoming challenges with these hybrid approaches is driving active research to address challenges in new applications.

This chapter provides an overview of the main concepts and principles of PINNs and introduces various methods for their applications with practical examples in mechatronic systems. Specifically, the remainder of this chapter is organized as

follows. Section 2 explains the fundamental principles and implementation of PINNs. Section 3 introduces the challenges and overcoming strategies in PINNs. In Section 4, the focus is on practical applications of PINNs in a complex mechatronic system. Finally, the chapter concludes with some remarks in Section 5.

2. Principles of physics-informed neural network

Physics-informed neural networks (PINNs) are designed to solve physical problems characterized by limited data availability, such as noisy measurements and time-consuming simulations [41]. These networks constrain the predicted solution to meet the laws of physics and fundamental principles to enhance predictive capabilities by overcoming a data scarcity problem. The physical laws are typically expressed through the governing equations, including parametrized and nonlinear partial differential equations (PDEs) in a generalized mathematical form [21] as

$$\frac{\partial u}{\partial t} + \mathcal{N}[u;\lambda] = 0, \quad x \in \Omega, t \in [0,T], \tag{1}$$

where u, $\mathcal{N}(u;\lambda)$, and Ω denote the hidden solution $u(t,x)$ depending on time t and spatial coordinates x, nonlinear differential operator parameterized by coefficient λ, and a subset of $\mathbb{R}^D$. This form can describe a variety of physics in nature and engineering system governed by dynamics, solid mechanics, thermodynamics, or fluid mechanics [21]. Some prominent examples include the Navier–Stokes equations elucidating fluid flows [36], the heat conduction equations for heat transfer [38], and Cauchy's equations of motion which depict solid deformation [42]. Therefore, this prior knowledge can be integrated into existing neural network models due to their mathematical representation, deriving the architecture of the PINNs.

The PINN architecture closely resembles a conventional MLP structure because the architecture comprises an input layer, several hidden layers, and an output layer (**Figure 1**). However, PINNs differ from conventional MLPs due to the placement of

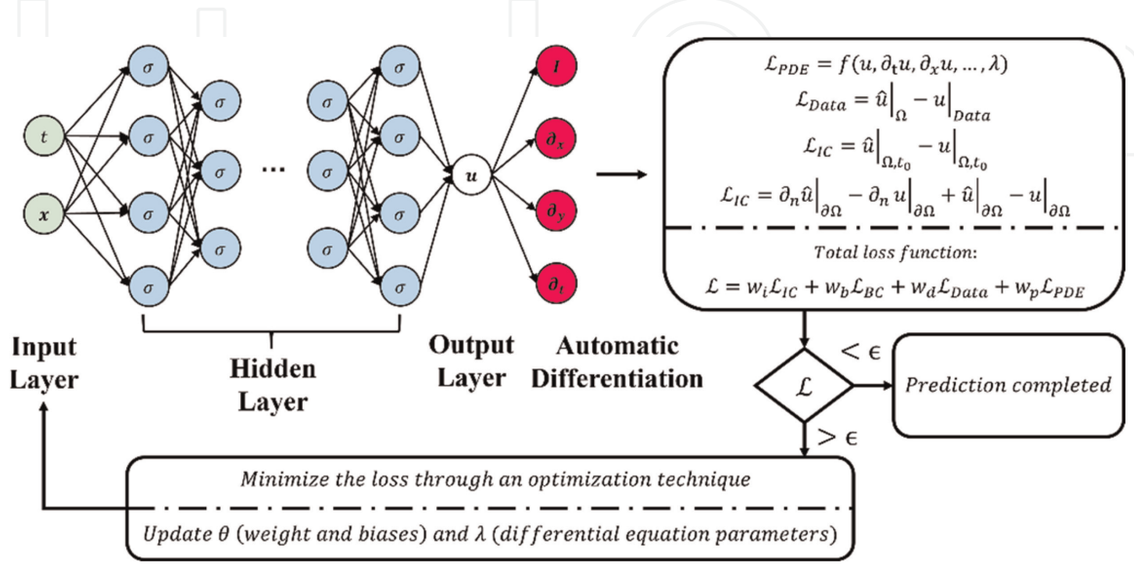


Figure 1.
A basic architecture of vanilla PINNs [43].

the differentiation layer (the red circles with differential symbols in **Figure 1**), right next to the output layer. Each of these layers plays a distinct and crucial role in the training process of PINNs. The spatiotemporal information, coordinates x and time t, are input to the neural network consisting of the hidden layers through the input layer. The neural network subsequently proceeds to predict the hidden solution u of the PDE through the output layers. The outputs u are directed into the subsequent differentiation layer. These layers perform automatic differentiation on neural networks with respect to both their input coordinates x and time t to construct each differential term of the PDE, resulting in the derivation of the loss function for network learning.

The loss function $\mathcal{L}$ of a PINN comprises the sum of two loss functions, $\mathcal{L}_{PDE}$ and $\mathcal{L}_u$ (as)

$$\mathcal{L} = \mathcal{L}_{PDE} + \mathcal{L}_u, \tag{2}$$

where $\mathcal{L}_{PDE}$ and $\mathcal{L}_u$ denote the loss function for penalization of the residual of the PDE, and loss function to satisfy the model predictions for data points [39]. Specifically, the first term $\mathcal{L}_{PDE}$ of $\mathcal{L}$ enforces the predicted solution $\hat{u}$ to obey the PDE at spatiotemporal points in a given computational domain [21] as

$$\mathcal{L}_{PDE} = \frac{1}{N_f}\sum_{i=1}^{N_f}\left|f\left(t_f^i, x_f^i\right)\right|^2, \tag{3}$$

where f denotes the residuals of the PDE expressed by the left-hand side of Eq. (1) at the collocation points t_f^i and x_f^i as

$$f\left(t_f^i, x_f^i\right) = \frac{\partial \hat{u}\left(t_f^i, x_f^i\right)}{\partial t} + \mathcal{N}\left[\hat{u}\left(t_f^i, x_f^i\right); \lambda\right]. \tag{4}$$

The second term $\mathcal{L}_u$ represents the sum of the loss functions aimed at ensuring the adherence of model predictions $\hat{u}$ to both initial and boundary conditions ($\mathcal{L}_{IC}$ and $\mathcal{L}_{BC}$), as well as labeled data ($\mathcal{L}_{data}$) as

$$\begin{aligned}\mathcal{L}_u &= \mathcal{L}_{IC} + \mathcal{L}_{BC} + \mathcal{L}_{data}\\ &= \frac{1}{N_{IC}}\sum_{i=1}^{N_{IC}}\left|\hat{u}\left(t_{IC}^i, x_{IC}^i\right) - u_{IC}^i\right|^2\\ &\quad + \frac{1}{N_{BC_1}}\sum_{i=1}^{N_{BC_1}}\left|\hat{u}\left(t_{BC_1}^i, x_{BC_1}^i\right) - u_{BC_1}^i\right|^2\\ &\quad + \frac{1}{N_{BC_2}}\sum_{i=1}^{N_{BC_2}}\left|\Gamma\left(\hat{u}_{BC_2}^i\right) - \Gamma_{BC_2}^i\right|^2\\ &\quad + \frac{1}{N_{data}}\sum_{i=1}^{N_{data}}\left|\hat{u}\left(t_{data}^i, x_{data}^i\right) - u_{data}^i\right|^2,\end{aligned} \tag{5}$$

where BC_1, BC_2, and Γ denote Dirichlet, Neumann boundary conditions, and a function with derivatives of $\hat{u}$ at boundary points. N and the symbol $\hat{}$ also denote the number of data points within each condition and the predicted solutions by the PINN.

$\left\{t_{IC}^i, x_{IC}^i, u_{IC}^i\right\}_{i=1}^{N_{IC}}$, $\left\{t_{BC_1}^i, x_{BC_1}^i, u_{BC_1}^i\right\}_{i=1}^{N_{BC_1}}$, $\left\{t_{BC_2}^i, x_{BC_2}^i, u_{BC_2}^i\right\}_{i=1}^{N_{BC_2}}$, and $\left\{t_{data}^i, x_{data}^i, u_{data}^i\right\}_{i=1}^{N_{data}}$ denote the initial training data, boundary training data of Dirichlet and Neumann conditions, and labeled data points. Note that the PINN's trainable parameters θ are updated via back propagation to minimize the loss function $\mathcal{L}$. During this optimization process, the ADAM optimizer [44] or L-BFGS optimizer [45] can be used.

To demonstrate the performance of PINN, the equation of motion of a 1-DOF mass-spring-damper system is used as a simple example. This equation is a linear ordinary differential equation (ODE) dependent on time t, describing the vibration of a mass. The governing equation is expressed as

$$\frac{d^2u}{dt^2} + 2\zeta\omega_n\frac{du}{dt} + \omega_n^2 u = 0, \quad t \in [0,1] \tag{6}$$

where u denotes the displacement, which is the ODE solution that PINN needs to predict. To this end, the PINN architecture uses temporal information, specifically time t, as network input and predicts the unknown solution u as output. The predicted solution u undergoes automatic differentiation within the differentiation layer to construct the loss function as Eqs. (3) and (4). Therefore, the total loss function for training PINN to solve the equation of motion is formulated as

$$\begin{aligned}\mathcal{L} = \mathcal{L}_{ODE} + \mathcal{L}_{Data} = &\frac{1}{N_f}\sum_{i=1}^{N_f}\left|\left(m\frac{d^2}{dt^2} + c\frac{d}{dt} + k\right)\hat{u}\left(t_f^i;\theta\right)\right|^2 \\ &+ \frac{1}{N_{data}}\sum_{i=1}^{N_{data}}\left|\hat{u}\left(t_{data}^i;\theta\right) - u_{data}^i\right|^2,\end{aligned} \tag{7}$$

where $\left\{t_f^i\right\}_{i=1}^{N_f}$ and $\left\{t_{data}^i, u_{data}^i\right\}_{i=1}^{N_{data}}$ denote the collocation points sampled within the range $t \in [0,1]$ and the training data points, including the initial conditions.

The PINN architecture for solving Eq. (6) consists of an input layer, multiple hidden layers, and an output layer as shown in **Figure 1**. The input layer has a single node, because it receives one-dimensional temporal information. The hidden layer consists of three layers, each containing 35 nodes and a hyperbolic tangent activation function. The output layer also comprises a single node to predict the displacement of a single DOF under free vibration conditions. To demonstrate the superiority of PINN, an artificial neural network (ANN) with identical architecture was constructed for comparison.

The training data points for both PINN and ANN were sampled from solutions of Eq. (6), with the damping ratio ζ set to 0.025 and the natural frequency ω_n to 20 rad/s. Specifically, the training dataset consisted of only five solutions, sampled at uniform time intervals. On the other hand, 30 collocation points were chosen for computing the physics loss in the PINN.

PINN and ANN were trained on the Colab environment with an Intel Xeon CPU and a Tesla T4. Both PINN and ANN were trained for 18,000 iterations, with identical hyper-parameters to ensure consistent comparisons except learning rates (**Table 1**).

Figure 2 shows the predicted solution by the trained PINN and ANN models, in comparison to the exact solution (the blue line in **Figure 2(a)** and **(b)**). Note that

	PINN	ANN
Number of input nodes	1	1
Number of hidden layers	3	3
Number of hidden nodes	35	35
Activation function	Tanh	Tanh
Number of output nodes	1	1
Learning rate	1e−4	1e−3
Epoch	18,000	18,000

Table 1.
Hyper-parameters of PINN and ANN.

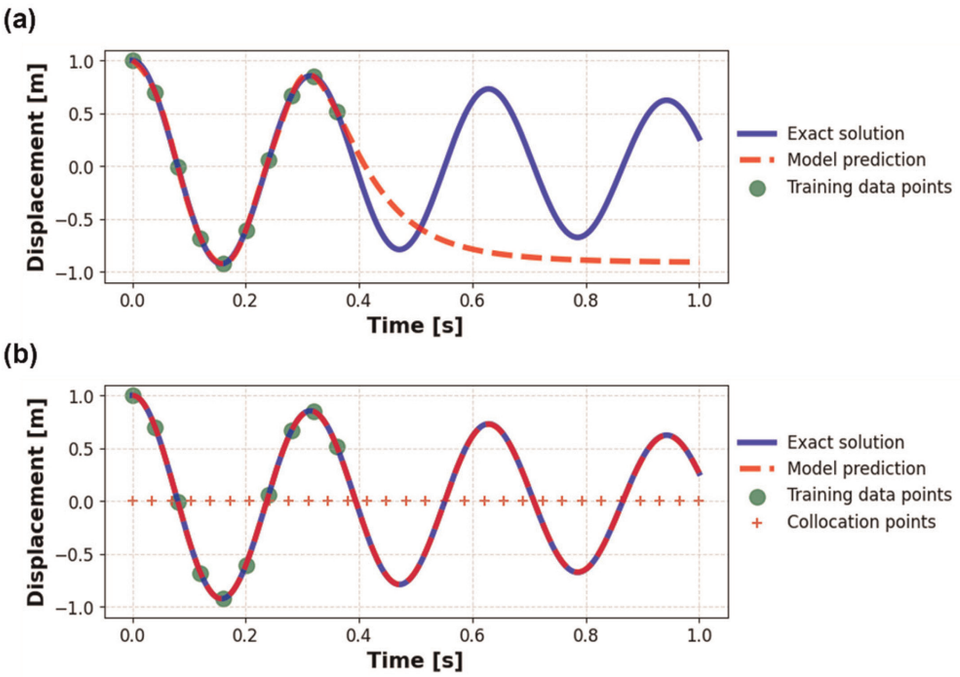


Figure 2.
Comparisons of the exact solution with the model prediction by ANN (a) and PINN (b).

ANN (the red dashed line in **Figure 2(a)**) accurately predicted only the data points on which it had been trained (the green circular marker in **Figure 2(a)**), failing to generalize to any other data. This result demonstrates the common issue of poor generalization performance in ANNs, where sparse training data limits the model's ability to learn the full spectrum of inherent physical properties, causing it to be biased toward only the observed data [43]. Therefore, the use of ANN in engineering and scientific field, where obtaining large amounts of training data is challenging [43], has significant limitations, suggesting the need for PINNs in scientific machine learning. **Figure 2(b)** confirms that the PINN accurately predicts the solution outside the training data region, outperforming the ANN with the same training data because the physical constraints embedded in the PINN guide the model predictions to conform to the underlying physical laws. This result suggests that PINN may exhibit superior generalization performance compared to ANN in real world applications where acquiring data is challenging.

3. Challenges and overcoming strategies in physics-informed neural networks

PINNs represent a significant advancement in the integration of scientific machine learning with physics, designed to model complex systems, where traditional data-driven approaches fall short. PINNs provide predictions that adhere to known physics and fundamental principles by embedding the governing equations of physical systems into neural networks. Hence, this architecture serves as an alternative to traditional data-driven methods for solving PDEs. However, a variety of challenges along with a series of promising results are being encountered as PINNs are actively researched. This section delves into these challenges and elucidates strategies to address them, thereby enhancing the robustness and versatility of PINNs across various scientific and engineering domains.

3.1 Mitigating complexity with domain decomposition

PINNs have shown remarkable abilities in addressing high-dimensional PDEs due to their superior approximation and generalization capabilities [47]. Despite this, PINNs can encounter difficulties when dealing with discontinuities in PDE solutions, often failing to accurately capture and predict sharp changes or discontinuities, which can lead to errors and reduce the accuracy of the solution [48]. To overcome these challenges, an innovative strategy known as extended PINN (XPINN) has been developed (**Figure 3**), which involves decomposing the computational domain into multiple subdomains [46]. This method employs localized, robust networks specifically in regions with high-gradient solutions, leveraging known prior knowledge about the solution characteristics. In particular, the computational domain can be partitioned into N_{sub} number of non-overlapping regular or irregular subdomains. Within the XPINN framework, the neural network's output for the kth subdomain is defined as

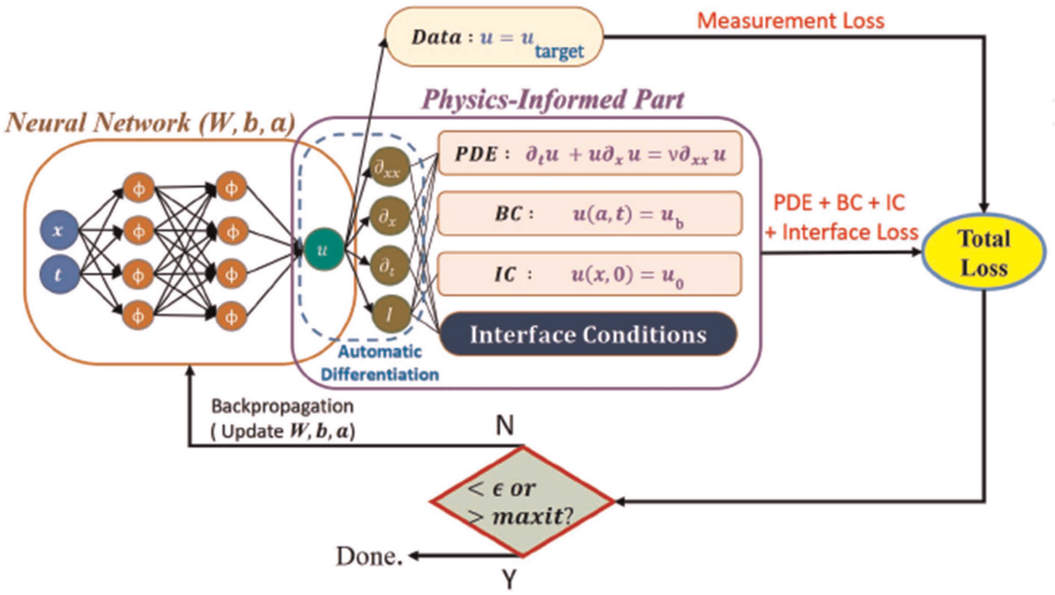


Figure 3.
The schematic of the XPINN sub-net employed in a subdomain, integrating neural network and physics-informed components [46].

$$u_{\tilde{\Theta}_k}(z) = N\left(z; \tilde{\Theta}_k\right) \in \Omega_k . \mathrm{k} = 1, 2, \ldots, N_{sub} \tag{8}$$

Then, the final solution is obtained as follows

$$u_{\tilde{\Theta}}(z) = \sum_{i=1}^{N_{u_k}} u_{\tilde{\Theta}_k}(z) \cdot \mathrm{I}_{\Omega_k}(\mathrm{z}), \tag{9}$$

where the indicator function $\mathrm{I}\Omega_k\ (\mathrm{z})$ is defined as

$$\mathrm{I}_{\Omega_k}(\mathrm{z}) = \begin{cases} 0, & \text{if } z \notin \Omega_k \\ 1, & \text{if } z \in \Omega_k \\ 1/S, & \text{if } z \text{ is on the interface within } \Omega_k \end{cases}, \tag{10}$$

where S denotes the number of subdomains intersecting along the interface. Each individual PINN operates independently within its assigned subdomain, thereby improving the network's capability to accurately capture and represent complex behaviors.

The XPINN framework leverages the deployment of multiple neural networks across smaller subdomains [49]. A key advantage of XPINNs is their enhanced representation and parallelization capacity, which is essential for accurately predicting complex PDE solutions. By enabling both spatial and temporal parallelization, XPINNs can significantly reduce training costs. In each subdomain, the networks can be optimally designed with specific hyperparameters such as network depth and width of hidden layers, the number and placement of residual points, nonlinear activation functions, and optimization strategies. For subdomains with complex solution characteristics, deeper networks may be employed, whereas shallower networks can be utilized in regions with simpler, smoother solutions.

The XPINN is designed to minimize a comprehensive loss function that considers both individual subdomain losses and interface losses, effectively balancing data accuracy and adherence to physical laws across the entire system. The XPINN loss function is defined at the subdomain level, while also incorporating interface conditions to ensure continuity and coherence between subdomains. Training an XPINN involves minimizing this complete loss function, which is defined as:

$$\begin{aligned} \mathcal{L}\left(\tilde{\Theta}_k\right) = W_{u_k}\mathcal{L}_{u_k}\left(\tilde{\Theta}_k; \left\{x_{u_k}^{(i)}\right\}_{i=1}^{N_{u_k}}\right) + W_{\mathcal{F}_k}\mathcal{L}_{\mathcal{F}_k}\left(\tilde{\Theta}_k; \left\{x_{\mathcal{F}_k}^{(i)}\right\}_{i=1}^{N_{\mathcal{F}_k}}\right) \\ + W_{I_k}\mathcal{L}_{u_{avg}}\left(\tilde{\Theta}_k; \left\{x_{I_k}^{(i)}\right\}_{i=1}^{N_{I_k}}\right) + W_{I_{\mathcal{F}_k}}\mathcal{L}_{\mathcal{R}}\left(\tilde{\Theta}_k; \left\{x_{I_k}^{(i)}\right\}_{i=1}^{N_{I_k}}\right) \end{aligned} \tag{11}$$

where $k = 1, 2, \ldots, N_{sub}$. The weight $W_{u_k}, W_{\mathcal{F}_k}, W_{I_k}$ and $W_{I_{\mathcal{F}_k}}$ correspond to the loss term for discrepancies in data, residual errors, and interface conditions, including both residual and average solution continuity at the interface, respectively. Each loss function is defined as:

$$\mathcal{L}_{u_k}\left(\tilde{\Theta}_k; \left\{x_{u_k}^{(i)}\right\}_{i=1}^{N_{u_k}}\right) = \frac{1}{N_{u_k}}\sum_{i=1}^{N_{u_k}}\left|u^i - u_{\Theta_k}\left(x_{u_k}^i\right)\right|^2 \tag{12}$$

$$\mathcal{L}_{\mathcal{F}_k}\left(\tilde{\Theta}_k; \left\{x_{\mathcal{F}_k}^{(i)}\right\}_{i=1}^{N_{\mathcal{F}_k}}\right) = \frac{1}{N_{\mathcal{F}_k}}\sum_{i=1}^{N_{\mathcal{F}_k}}\left|\mathcal{F}_{\tilde{\Theta}_k}\left(x_{\mathcal{F}_k}^i\right)\right|^2 \tag{13}$$

$$\mathcal{L}_{u_{avg}}\left(\tilde{\Theta}_k; \left\{x_{I_k}^{(i)}\right\}_{i=1}^{N_{I_k}}\right) = \sum_{\forall k^+}\left(\frac{1}{N_{I_q}}\sum_{i=1}^{N_{I_k}}\left|u_{\tilde{\Theta}_k}\left(x_{I_k}^i\right) - \left\{\left\{u\left(\boldsymbol{x}_{I_k}^i\right)\right\}\right\}\right|^2\right) \tag{14}$$

$$\mathcal{L}_{\mathcal{R}}\left(\tilde{\Theta}_k; \left\{x_{I_k}^{(i)}\right\}_{i=1}^{N_{I_k}}\right) = \sum_{\forall k^+}\frac{1}{N_{I_k}}\sum_{i=1}^{N_{I_k}}\left|\mathcal{F}_{\tilde{\Theta}_k}\left(x_{I_k}^i\right) - \mathcal{F}_{\tilde{\Theta}_{k^+}}\left(x_{I_k}^i\right)\right|^2 \tag{15}$$

where terms $\mathcal{L}_{u_k}$ and $\mathcal{L}_{\mathcal{F}_k}$ represent the data mismatch and residual losses, respectively, and $\mathcal{F}_{\tilde{\Theta}_k}$ denotes the PDEs residual terms of each subdomain. The residual loss term $\mathcal{L}_{\mathcal{R}}$ ensures residual continuity at the interface region between separate networks in subdomains k and k^+, where the superscript $^+$ indicates neighboring subdomain(s). Both $\mathcal{L}_{\mathcal{R}}$ and $\mathcal{L}_{u_{avg}}$ are defined for all neighboring subdomain k^+, as indicated by the summation over all k^+ in their respective expressions. These interface conditions enable the transfer of information between neighboring subdomains, playing a vital role in the convergence and overall accuracy of the subdomain solutions.

The XPINN architecture simplifies complex PDE solutions by decomposing them into multiple simpler components, thereby reducing the complexity required to learn each part and enhancing generalization. XPINNs also provide flexibility through time-domain decomposition for all differential equations and the ability to manage arbitrarily shaped complex sub-regions. This methodology ensures robust and accurate estimation of responses and other complex phenomena in systems modeled by PINNs.

3.2 Loss balancing with adaptive weighting

Most PINNs encounter a common convergence issue due to loss function imbalance, which arises from different optimization rates of multiple loss functions. Specifically, each of these functions contributes to the overall training objective but may have vastly different scales and sensitivities. For example, one loss function might represent the governing equation in thermodynamics, while another could enforce an initial condition. The different complexity in training these loss functions can lead to discrepancies in the magnitudes of optimizing gradients during training. This discrepancy often results in a situation where optimizing one loss function significantly overshadows the others, thereby stalling the overall convergence of the network.

To overcome this challenge, adaptive weighting methods are being actively researched, including self-adaptive [26] and learning rate annealing [28]. These methods achieve convergence by adjusting the gradient differences derived from each loss function. In particular, adaptive weighting using the neural tangent kernel (NTK) [50], a widely used method, addresses the optimization imbalance caused by multiple loss functions. This strategy uses NTK to dynamically adjust adaptive weights during training, guaranteeing that all loss terms converge equitably and thereby preserving overall balance. The standard loss function of PINNs with adaptive weights is as follows:

$$L(\theta, \lambda_D, \lambda_P, \lambda_B, \lambda_I) = \lambda_D L_D(\theta) + \lambda_P L_P(\theta) + \lambda_I L_I(\theta) + \lambda_B L_B(\theta) \tag{16}$$

where θ, L_D, L_P, L_B, L_I, and λ_* represent the trainable parameters, data fitting loss, physics loss, boundary condition loss, initial condition loss, and adaptive weights for each loss function, respectively. Note that the vanilla loss function of PINNs is the same as when λ_* is 1. Note also that there may not be a specific loss function

depending on the problem being solved using PINN. To determine the adaptive weights for each loss in Eq. (16), the NTK of PINN $H^{NTK}(\theta)$ is defined as follows:

$$H_{ij}^{NTK}(\theta_n) = \left\langle \frac{dT^i(\theta_n)}{d\theta_n}, \frac{dT^j(\theta_n)}{d\theta_n} \right\rangle, \quad i,j = 1,2,\cdots,N,$$
$$T^k(\theta_n) = \begin{cases} X^k - \hat{X}^k, 1 \le \mathrm{k} \le N_D \\ \widehat{Res_P}^k, N_D < \mathrm{k} \le N_D + N_P \\ \widehat{Res_{BC}}^k, N_D + N_P < \mathrm{k} \le N_D + N_P + N_B \\ {X_0}^k - \widehat{X_0}^k, N_D + N_P + N_B < \mathrm{k} \le N \end{cases}, \tag{17}$$

where θ_n denotes parameters of the PINN at the n-th iteration, and $N = N_D + N_P + N_I + N_B$ represents the total collocation points considering data fitting, physics, initial condition, and boundary condition. Then, λ^k, representing the adaptive weights of the k-th collocation point, is obtained as follows:

$$\lambda_\psi = \frac{\sum_{k=1}^{N} H_{kk}^{NTK}(\theta_n)}{\sum_{k=1}^{N_\psi} H_{kk}^{NTK}(\theta_n)}, \psi \in \{D,P,B,I\} \tag{18}$$

where H_{kk}^{NTK} represents the k-th diagonal entry of the NTK matrix, which indicates the backpropagation gradient of the loss function. This balancing strategy is crucial because unbalanced gradients can significantly hinder the optimization process, preventing the network from finding a solution that satisfies the imposed physical constraints.

3.3 Extending capabilities through deep neural operator

In real-world applications, engineers often face the challenge of predicting outcomes under diverse operating conditions that differ from the specific conditions in which the neural network was trained. However, conventional PINNs that use DNNs are typically designed to train and predict based on specific initial, boundary, or geometric conditions. These inherent characteristics necessitate independent training processes, such as retraining or transfer learning [51], to adapt to new operating conditions. This additional training is not only time-consuming but also computationally expensive, limiting its use as a surrogate model. Therefore, significant research efforts have been directed toward developing more capable neural network to address these challenges.

Deep Operator Network (DON) has become a state-of-the-art surrogate model due to its ability to approximate nonlinear operators through function-to-function mapping [31]. Hence, once DON is trained, it can predict results for new operational conditions, effectively addressing both interpolation and extrapolation tasks without additional retraining. Specifically, DON integrates two distinct neural networks (**Figure 4**): the branch and trunk networks. The branch network handles discretized inputs from the input functions, such as initial conditions, boundary conditions, or external forcing terms, represented as $\boldsymbol{u} = [u(t_1), u(t_2), u(t_3), \cdots, u(t_{N-1}), u(t_N)]$, whereas the trunk network uses spatiotemporal coordinates $\boldsymbol{y} \in \mathbb{R}^d$ as inputs. The branch network is responsible for encoding

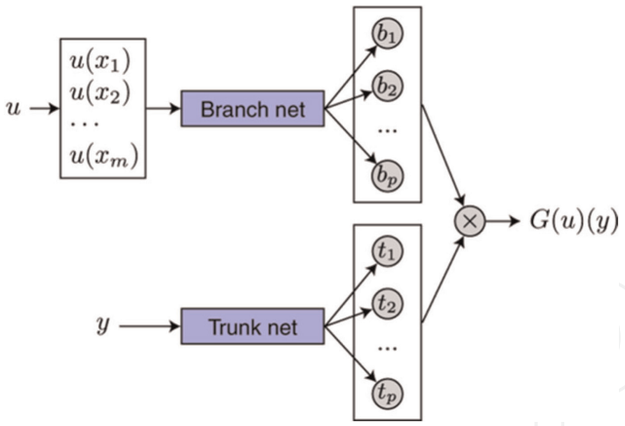


Figure 4.
Graphical illustration of DON [31].

the input function at specific points within the domain. This process is crucial for enabling the network to serve as a surrogate model under varying specific initial or boundary conditions. Simultaneously, the trunk network is designed to handle the spatial temporal coordinates for values of interest. Then, the outputs of both networks are integrated through a dot product operation, such that $G(\boldsymbol{u})(\boldsymbol{y})$ becomes a function of $\boldsymbol{y}$ on $\boldsymbol{u}$. Specifically, the outputs from both networks are obtained by forward propagation through an L-hidden layer DON based on a multilayer perceptron, as detailed below. The hidden layer outputs $\boldsymbol{H}_u$ and $\boldsymbol{H}_y$ for the branch network and the trunk network are obtained as follows:

$$\boldsymbol{H}_u^{(1)} = \varnothing\left(\boldsymbol{W}_u^{(1)}\boldsymbol{u} + \boldsymbol{b}_u^{(1)}\right), \boldsymbol{H}_y^{(1)} = \varnothing\left(\boldsymbol{W}_y^{(1)}\boldsymbol{y} + \boldsymbol{b}_y^{(1)}\right), \tag{19}$$

$$\boldsymbol{H}_u^{(l)} = \varnothing\left(\boldsymbol{W}_u^{(l)}\boldsymbol{H}_u^{(l-1)} + \boldsymbol{b}_u^{(l)}\right), \boldsymbol{H}_y^{(l)} = \varnothing\left(\boldsymbol{W}_y^{(l)}H_y^{(l-1)} + \boldsymbol{b}_y^{(l)}\right), \; l = 2, \cdots, L-1, \tag{20}$$

$$\boldsymbol{O}_u = \boldsymbol{H}_u^{(L)} = \varnothing\left(\boldsymbol{W}_u^{(L)}\boldsymbol{H}_u^{(L-1)} + \boldsymbol{b}_u^{(L)}\right), \boldsymbol{O}_y = \boldsymbol{H}_y^{(L)} = \varnothing\left(\boldsymbol{W}_y^{(L)}H_y^{(L-1)} + \boldsymbol{b}_y^{(L)}\right), \tag{21}$$

where $\varnothing$ represents an activation function and $\left\{\boldsymbol{W}_u^{(l)}, \boldsymbol{b}_u^{(l)}\right\}_{l=1}^{L}$ and $\left\{\boldsymbol{W}_y^{(l)}, \boldsymbol{b}_y^{(l)}\right\}_{l=1}^{L}$ refer to the weights and biases of the hidden layers for the branch network and the trunk network, respectively. In both networks, the initial hidden layers take $\boldsymbol{u}$ and $\boldsymbol{y}$ as inputs, whereas following hidden layers utilize the outputs of the previous hidden layer $\boldsymbol{H}_u^{(l-1)}$ and $\boldsymbol{H}_y^{(l-1)}$. The inputs $\boldsymbol{u}$ and $\boldsymbol{y}$ are converted to output vectors $\left[O_u^1, O_u^2, O_u^3, \cdots, O_u^{q-1}, O_u^q\right]^T \in \mathbb{R}^q$ and $\left[O_y^1, O_y^2, O_y^3, \cdots, O_y^{q-1}, O_y^q\right]^T \in \mathbb{R}^q$ through the branch and trunk networks, respectively. Finally, the neural network predictions are derived by integrating the output vectors from the branch and trunk networks as follows:

$$G(\boldsymbol{u})(\boldsymbol{y}) \approx \hat{G}(\boldsymbol{u})(\boldsymbol{y}) = \sum_{k=1}^{q} \underbrace{O_u^k(u(t_1), u(t_2), \cdots, u(t_N))}_{branch} \underbrace{O_y^k(\boldsymbol{y})}_{trunk} \tag{22}$$

where $\hat{G}(\boldsymbol{u})(\boldsymbol{y})$ denotes a predicted DON value, while $G(\boldsymbol{u})(\boldsymbol{y})$ corresponds to the ground truth value.

The capability of DON to capture operators can be leveraged as a powerful surrogate model for multiphysics systems governed by a range of ODEs and PDEs. Therefore, active research has been directed toward predicting responses under a variety of conditions within the fields of elastic-plastic stress [52], heat conduction [53], and fracture mechanics [54]. Furthermore, numerous studies are dedicated to improving the capabilities of DON through architectural advancements, integrating neural networks such as encoders [55], LSTM [56], or ResUNet [52].

3.4 Improving architecture through Fourier feature embedding

PINNs often face challenges when addressing multi-scale problems. PINNs operate with a relatively limited number of data points within the computational domain. One significant challenge is spectral bias, a phenomenon where multi-layer perceptron tends to train solutions and their PDE residuals starting with the lowest frequency components. Spectral bias is an intriguing characteristic observed in the neural networks training, where these networks tend to learn simpler or lower-frequency components before capturing more complex or higher-frequency components of the target function [57]. This phenomenon can significantly influence the training dynamics and generalization abilities of neural networks.

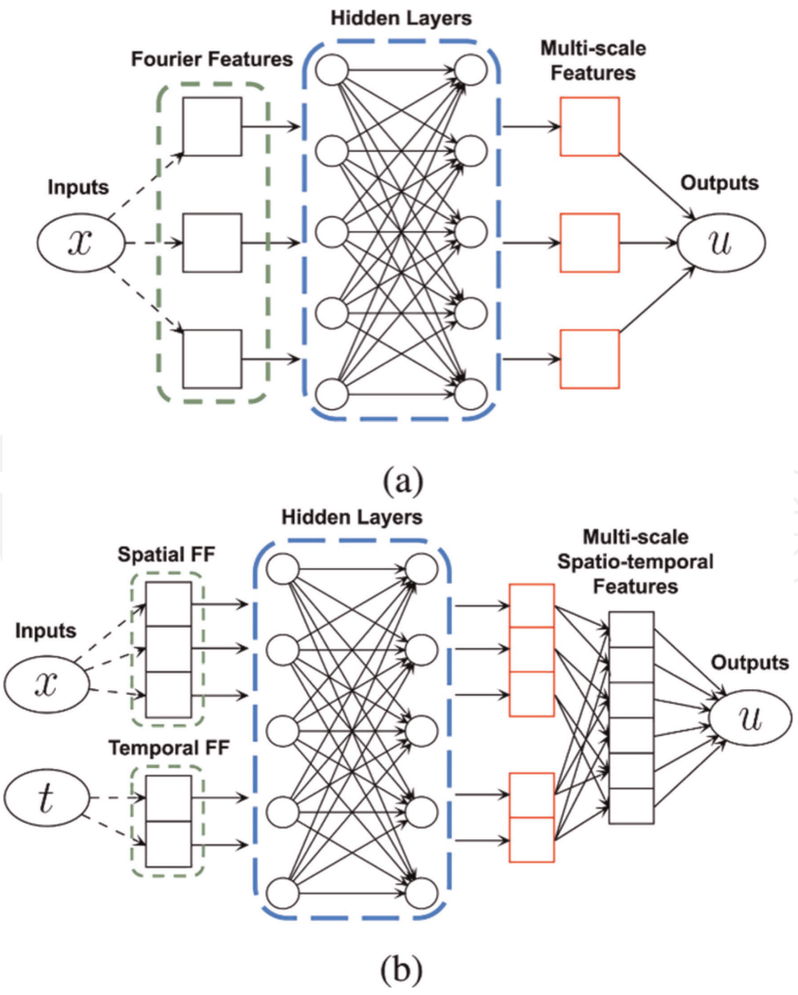


Figure 5.
(a) Multi-scale Fourier feature network, and (b) Spatio-temporal multi-scale Fourier feature network [57].

To address the multi-scale problem challenges in PINNs, particularly spectral bias, an innovative network architecture incorporating Fourier feature embeddings has been developed [33]. This approach involves enhancing the input space using Fourier features, which helps the network represent and learn higher frequency components more effectively.

As illustrated in **Figure 5(a)**, the methodology involves applying Fourier feature embeddings. The inputs are embedded with the Fourier feature and then fed into a conventional multi-layer perceptron. The final outputs are concatenated using a linear function. The forward pass of Fourier feature embedding is detailed as follows:

$$\boldsymbol{\alpha}^{\boldsymbol{i}}(\boldsymbol{x}) = \begin{bmatrix} \cos\left(2\pi \boldsymbol{F}^i \boldsymbol{x}\right) \\ \sin\left(2\pi \boldsymbol{F}^i \boldsymbol{x}\right) \end{bmatrix}, \quad for\ i = 1, 2, \ldots, M, \tag{23}$$

$$\boldsymbol{H}_1^i = \phi\left(\boldsymbol{W}_1 \cdot \boldsymbol{\alpha}^j(\boldsymbol{x}) + \boldsymbol{b}_1\right), \quad for\ i = 1, 2, \ldots, M, \tag{24}$$

$$\boldsymbol{H}_l^i = \phi\left(\boldsymbol{W}_l \cdot \boldsymbol{H}_{l-1}^j(\boldsymbol{x}) + \boldsymbol{b}_l\right), \quad for\ l = 2, \ldots, i = 1, 2, \ldots, M, \tag{25}$$

$$f_\theta(\boldsymbol{x}) = \boldsymbol{W}_{L+1}\left[\boldsymbol{H}_L^1, \ldots, \boldsymbol{H}_L^M\right] + \boldsymbol{b}_{L+1} \tag{26}$$

where ϕ and $\boldsymbol{\alpha}^i$ represent nonlinear activation functions and Fourier feature mappings, respectively. Each $\boldsymbol{F}^i$ is sampled from the normal distribution $\mathcal{N}\left(0, \sigma^i\right)$, and remains fixed during model training, meaning that $\boldsymbol{F}^i$ are not trainable parameters [33]. The inputs are embedded with different frequencies, and after forward propagation, all frequency components are combined through a linear layer to ensure they are learned at the same convergence rate [58].

Time-dependent problems exhibit multi-scale behavior not only across spatial dimensions but also over temporal scales. To address these multi-scale challenges in spatial-temporal domains, an innovative multi-scale Fourier feature architecture has been proposed [59]. Specifically, the neural network's feed-forward pass is defined as follows

$$\boldsymbol{\alpha}_s^i(\boldsymbol{x}) = \begin{bmatrix} \cos\left(2\pi \boldsymbol{F}_s^i \boldsymbol{x}\right) \\ \sin\left(2\pi \boldsymbol{F}_s^i \boldsymbol{x}\right) \end{bmatrix}, \boldsymbol{H}_{t,1}^i = \phi\left(\boldsymbol{W}_1 \cdot \boldsymbol{\alpha}_t^i(t) + \boldsymbol{b}_1\right), \quad for\ i = 1, 2, \ldots, M_s, \tag{27}$$

$$\boldsymbol{\alpha}_t^j(t) = \begin{bmatrix} \cos\left(2\pi \boldsymbol{F}_t^j t\right) \\ \sin\left(2\pi \boldsymbol{F}_t^j t\right) \end{bmatrix}, \boldsymbol{H}_{s,1}^i = \phi\left(\boldsymbol{W}_1 \cdot \boldsymbol{\alpha}_t^j(\boldsymbol{x}) + \boldsymbol{b}_1\right), \quad for\ j = 1, 2, \ldots, M_t, \tag{28}$$

$$\boldsymbol{H}_{s,l}^i = \phi\left(\boldsymbol{W}_l \cdot \boldsymbol{H}_{s,l-1}^i + \boldsymbol{b}_l\right), \quad for\ i = 1, 2, \ldots, M_x\ and\ l = 2, \ldots, L, \tag{29}$$

$$\boldsymbol{H}_{t,l}^j = \phi\left(\boldsymbol{W}_l \cdot \boldsymbol{H}_{t,l-1}^j + \boldsymbol{b}_l\right), \quad for\ i = 1, 2, \ldots, M_t\ \ and\ l = 2, \ldots, L. \tag{30}$$

$$\boldsymbol{H}_L^{i,j} = \boldsymbol{H}_{s,L}^i \odot \boldsymbol{H}_{t,L}^i, \tag{31}$$

$$f_\theta(\boldsymbol{x}, t) = \boldsymbol{W}_{L+1}\left[\boldsymbol{H}_L^{1,1}, \ldots, \boldsymbol{H}_L^{M_t, M_x}\right] + \boldsymbol{b}_{L+1}. \tag{32}$$

where ${\boldsymbol{\alpha}_x}^i$ and ${\boldsymbol{\alpha}_t}^i$ represent spatial and temporal Fourier feature mappings, respectively, with $\odot$ denoting the point-wise multiplication. Each entry of $\boldsymbol{F}_x^i$ and $\boldsymbol{F}_t^j$

is sampled from $\mathcal{N}(0, \sigma_x^i)$ and $\mathcal{N}\left(0, \sigma_y^j\right)$, respectively, and these values are kept constant during the training of the model.

This structured approach ensures that the PINN can manage the natural low-frequency bias of neural networks and is capable of effectively learning and predicting high-frequency phenomena within the PDEs solutions. By embedding Fourier features directly into the input layer and merging them before producing the final output, the network utilizes the rich frequency information to achieve more precise and robust solutions for PDE. This architecture significantly improves the ability of PINNs to solve complex, multi-scale problems that are typically challenging for conventional machine learning models and some numerical methods. Note that the proposed models maintain computational efficiency by avoiding the introduction of additional trainable parameters and not requiring significantly more operations for their forward or backward pass compared to conventional PINN models.

4. Case studies in engineering applications

The potential of PINNs is actively being explored for real-world engineering applications. To address the inherent multiphysics nature of most real-world systems, PINNs should be extended to account for multiphysics in the system of interest [35]. However, incorporating multiphysics into a neural network increases the complexity, leading to convergence issues due to the more challenging training process. Additionally, having a comprehensive understanding of the target system and employing suitable strategies are crucial to ensuring optimal performance. This section presents two examples, where PINNs are applied to complex mechatronics systems, specifically involving a motor and a battery, by utilizing appropriate training strategies.

4.1 Application on an electric motor for virtual sensing and PHM

This subsection introduces the novel MPINN framework for virtual sensing and prognostics and health management (PHM) of electric motors [40]. The proposed framework is designed to accurately estimate electromagnetic responses in electric motors under various operational conditions. This MPINN utilizes novel strategies to manage complexities associated with rotational dynamics, improving both the accuracy and efficiency of network's predictions. The MPINN specifically estimates various electromagnetic fields, including magnetic vector potentials (A_z), electric fields (E_z), magnetic flux densities (B_x and B_y), and magnetic fields (H_z).

The proposed MPINN architecture integrates three crucial features to overcome the challenges of conventional networks. First, the MPINN utilizes various PDEs to describe the electromagnetic properties of motors [60], ensuring precise and reliable predictions even with limited data availability. These PDEs guide the neural network, providing supervision for accuracy. A notable innovation is the conversion of the motor's rotor coordinates from rotational (based on angular speed) to Cartesian coordinates. This transformation overcomes the limitations of conventional PINNs, which have difficulty with rotational machinery due to their fixed coordinate systems. By aligning the rotor and stator coordinates into a unified Cartesian framework, the MPINN effectively resolves component representation mismatches and accurately depicts the electromagnetic fields influenced by rotor movement. Second, the

architecture incorporates distinct networks for each domain, integrating estimation results by considering the interface loss between the two sperate domains [49]. This approach allows for more targeted training on specific components, significantly enhancing the accuracy of electromagnetic response estimations. Additionally, an interface loss function merges the outputs from these separate networks, reducing discrepancies and errors at the interface between two domains. Third, the MPINN guarantee convergence of neural networks during training by implementing a learning rate annealing method. This method adjusts adaptive weights to balance the gradient stiffness across different loss types, thereby minimizing the overall loss [25]. This enhancement not only improves the accuracy and reliability of predictions but also makes the neural network more robust in managing the complexities associated with different loss functions.

The proposed MPINN was validated under five conditions, as detailed in **Table 2**. These conditions include four typical operating scenarios of the PMSM and one fault condition involving rotor axis eccentricity, which is the most common mechanical defect [61]. The MPINN was trained using data generated from a Finite Element Model (FEM) that covers the entire domain of the PMSM. The performance of MPINN was verified by comparing torques measured from an experimental PMSM testbed. Additionally, comparisons with the FEM confirmed the accuracy of electromagnetic characteristics across the entire domain [62, 63].

The accuracy of the MPINN for the electric motor was validated by comparing the experimental measurements from the those estimated by the FEM and MPINN. The torque measurements are only compared because measuring electromagnetic fields directly through the experiments is difficult, and only torque can be obtained. As illustrated in **Figure 6**, the torques predicted by the FEM (red line) and MPINN (blue line) are compared with those obtained from the experiments (black line) across the five different operational scenarios. Both models predicted the dynamic PMSM's torques accurately when controlling the motor driver's three-phase currents and rotor rotational speed. The average RMSEs for the FEM and MPINN were 0.0105 Nm and 0.0166 Nm, respectively, demonstrating the accurate estimation of the PMSM's electromagnetic responses by both models. Furthermore, these results highlight the potential of MPINN in effectively managing complex and nonlinear behaviors in data, showcasing its applicability in various scenarios.

The effectiveness of the proposed MPINN in predicting electromagnetic fields across the entire domain was demonstrated by comparing its results with those obtained from FEM. The prediction of electromagnetic fields from the FEM and MPINN is shown in **Figure 7**, along with the error of the proposed MPINN compared to the FEM under eccentricity fault conditions. Specifically, the RMSEs are 8.78e−5

Case	Description	Condition	Load	Speed [RPM]
1	Starting,no load	Normal	—	0 → 60
2	Variational speed, no load	Normal	—	60 → 600 → 60
3	Constant speed, linearly increasing load	Normal	0 to 0.4 Nm (linear increase)	60
4	Breaking	Normal	Brake	60 → 0
5	Starting, no load, eccentricity	Eccentricity	—	0 → 60

Table 2.
Experimental datasets under various operational conditions [40].

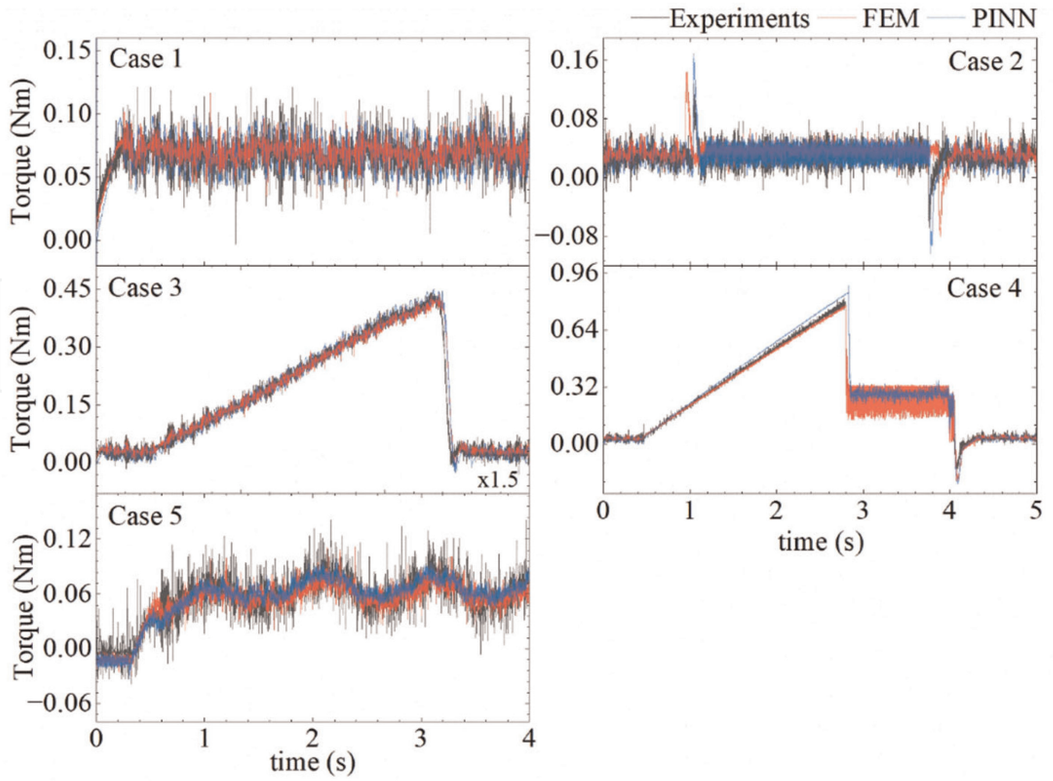


Figure 6.
Comparison of experimentally torque measurements with estimation results of FEM and MPINN across five different conditions [40].

for the magnetic vector potential, 4.86e−2 for the electric field, 5.64e−2 and 1.29e−1 for the magnetic flux density in the x and y directions, respectively, and 3.56e4 and 3.95e4 for the magnetic field in the x and y directions, respectively. These results show that the MPINN accurately estimates electromagnetic fields for the entire PMSM domain. Notably, errors in the electric field were observed across the entire domain due to a higher number of spatial training points compared to temporal ones, while errors in the magnetic flux density were confined to specific areas. This difference can be explained by the neural networks' tendency to provide more accurate estimates where training data points are more abundant.

This study highlights the superior performance of MPINN, which integrates physical constraints, compared to a conventional ANN lacking such constraints. As illustrated in **Figure 8**, the comparison of RMSEs for the electromagnetic responses predicted by the MPINN and the conventional ANN against the values derived from FEM. These results indicate that the RMSEs for the MPINN are consistently lower than those for the ANN in tested scenarios. Compared to the ANN, the MPINN achieves notable reductions in average RMSEs for the estimated values: 21.64% for the magnetic vector potential, 80.09% for the electric field, 70.91% and 39.44% for the magnetic flux densities in the x and y directions, respectively, and 58.39% and 53.83% for the magnetic fields in the x and y directions, respectively. These comparisons highlight the advantages of incorporating physical regularizations into the neural networks, enhancing the accuracy and robustness of estimation results compared to purely data-driven methods.

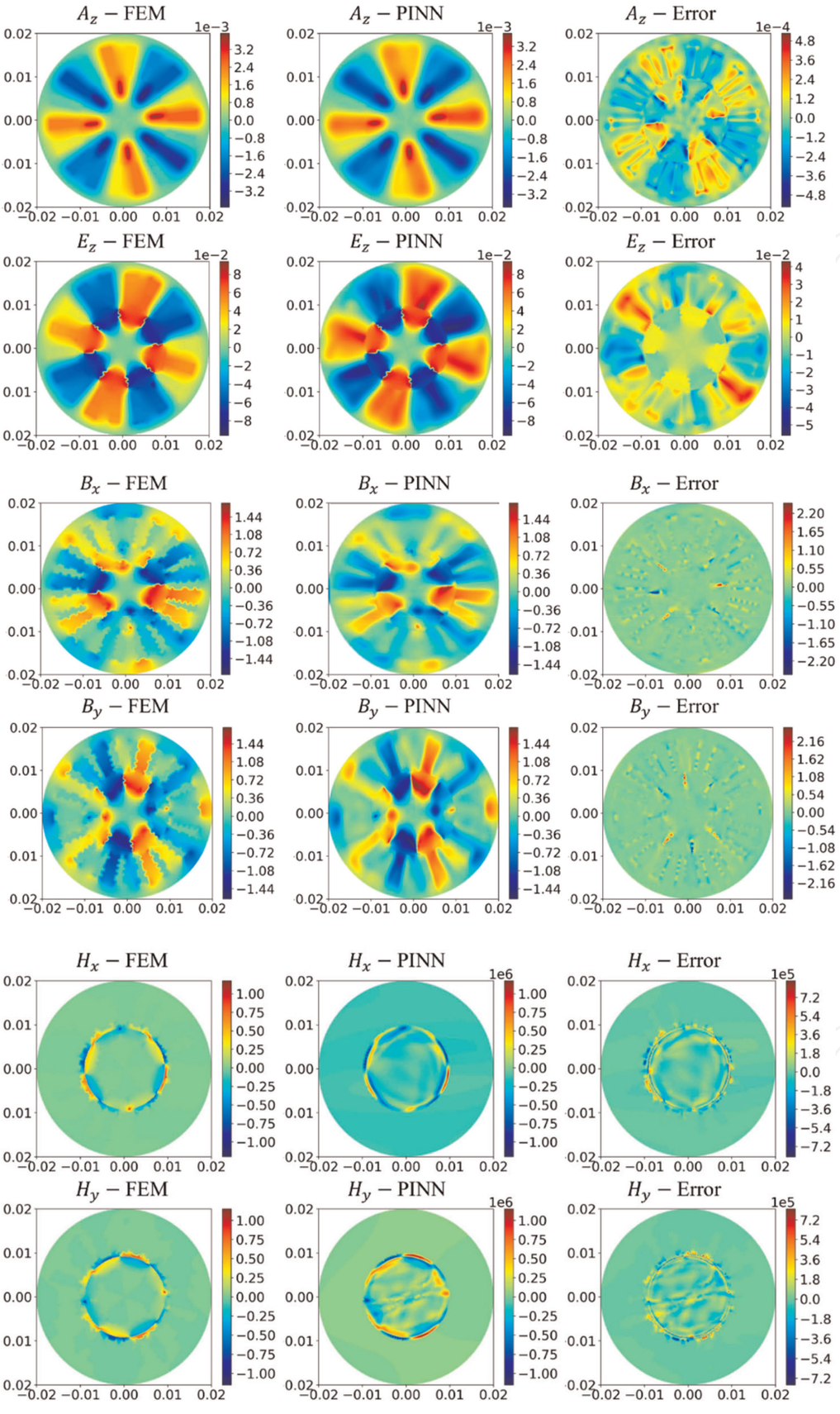


Figure 7.
Comparison of reference and estimated electromagnetic fields across eccentricity fault condition [40].

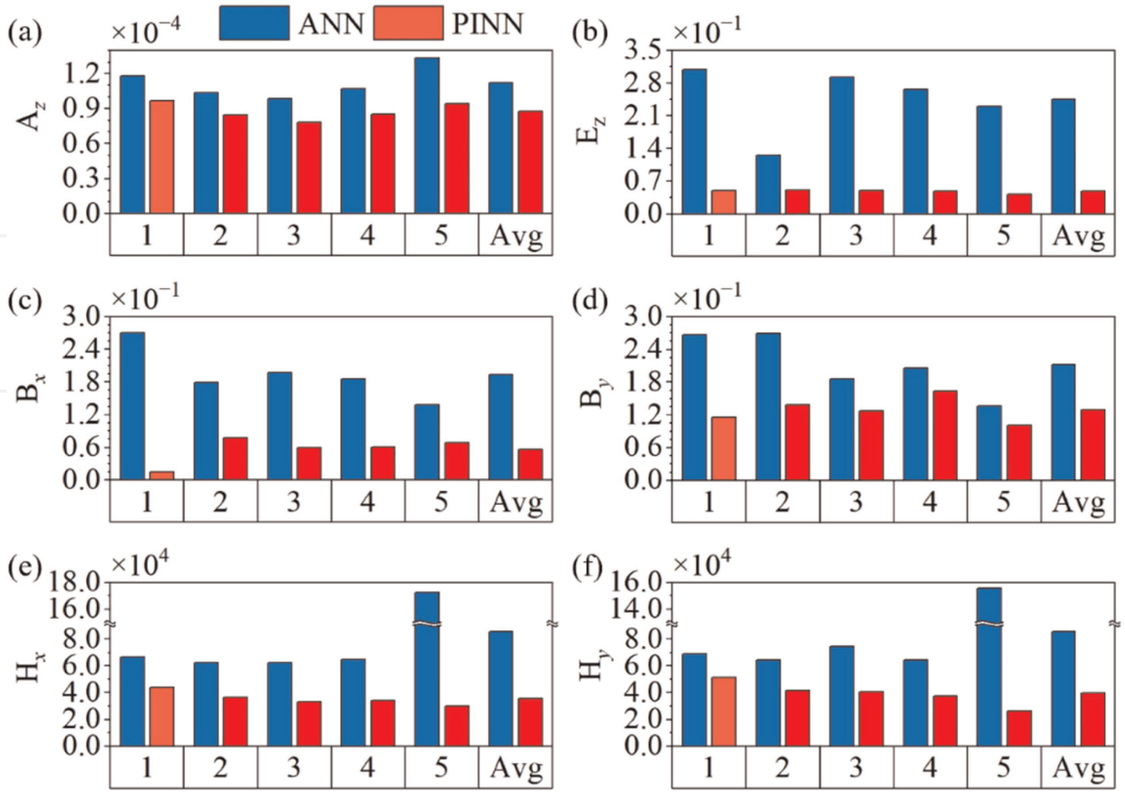


Figure 8.
RMSE comparison between ANN and MPINN estimation: (a) magnetic vector potential $\boldsymbol{A}_z$, magnetic flux densities (b) $\boldsymbol{B}_x$ and (c) $\boldsymbol{B}_y$, (d) electric field $\boldsymbol{E}_z$, and magnetic fields (e) $\boldsymbol{H}_x$, and (f) $\boldsymbol{H}_y$ [40].

Notably, as shown in **Figure 8(b)**, the MPINN could accurately estimate electromagnetic responses even when operational conditions changed significantly over time. In contrast, the purely data-driven approach faced difficulties in accurately capturing these variations. This highlights the effectiveness of incorporating physical principles, such as the temporal differentiation of the electric field and the magnetic vector potential via the PDE. This sharply contrasts with the ANN, which frequently violates physical laws due to having less data available compared to the network's capacity. Additionally, the distorted magnetic fields from asymmetric rotation pose significant challenges for the ANN when estimating electromagnetic responses under eccentric conditions. Consequently, the ANN is constrained in its ability to account for all intricate phenomena occurring in the PMSM under these operational conditions. Conversely, the MPINN, guided by the governing equation that defines the relationship between inputs and outputs, achieves robustness of estimation results. These analyses indicate that incorporating prior knowledge is crucial for PINN estimation. With the same dataset size, MPINNs enhance accuracy and ensure high reliability compared to conventional data-driven methods.

This research highlights that the proposed MPINN is a powerful and reliable solution for analyzing electromagnetic responses of PMSMs. While conventional FEM are accurate, they require substantial computational resources, and physical experiments typically yield data only for partial domains. In contrast, MPINNs offer a cost-effective alternative with rapid and accurate electromagnetic characterization capabilities. The MPINN not only addresses the limitations of conventional numerical methods by offering a mesh-free solver approach but also proves more efficient and cost-effective for characterizing electromagnetic properties in both normal and fault conditions.

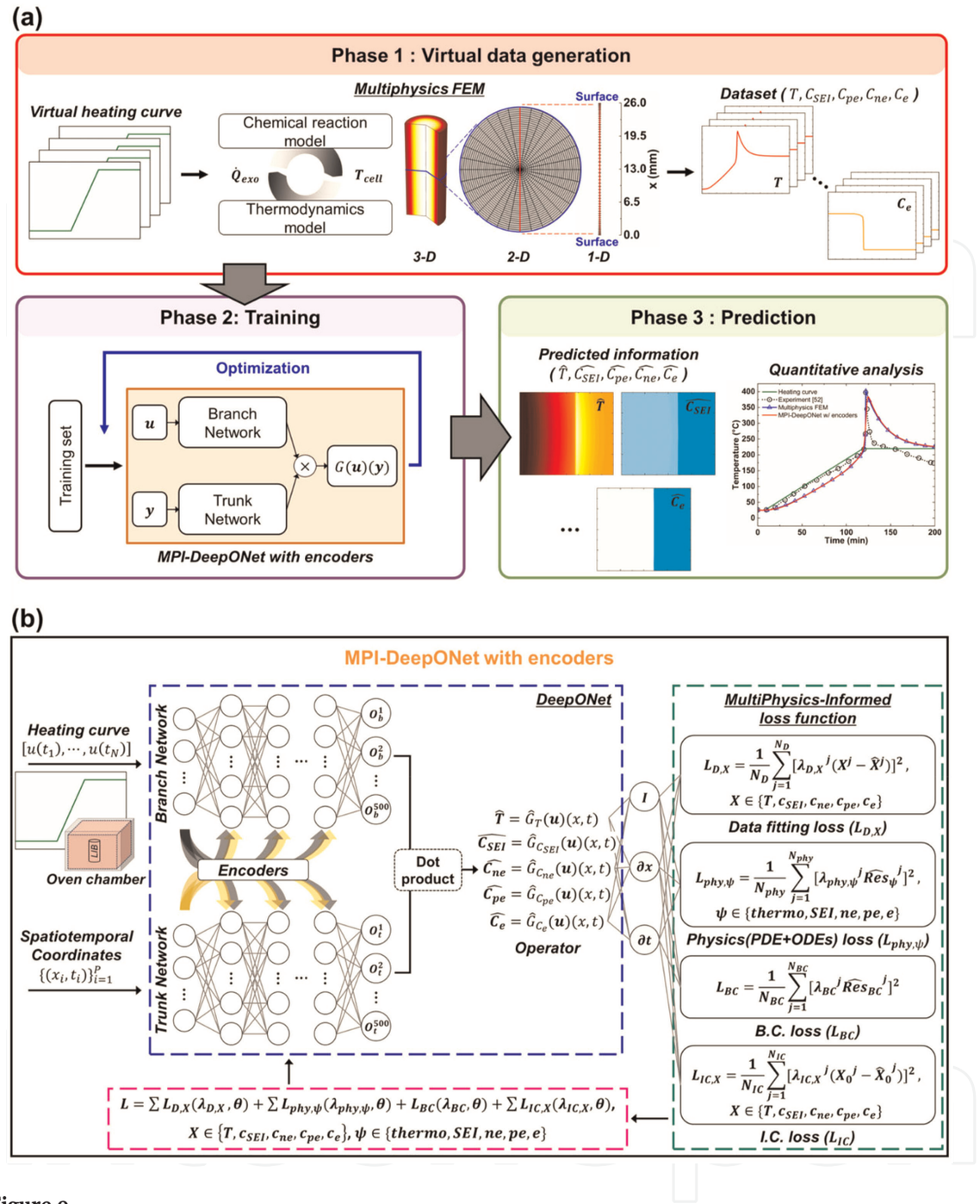


Figure 9.
(a) Flowchart for TR prediction, and (b) in-depth illustration of the MPI-DON incorporating encoders [37].

This makes MPINNs particularly suitable for PHM applications in electric vehicles and cyber-physical systems in production facilities, where control-oriented objectives are crucial.

4.2 Application for predicting thermal runaway of a lithium-ion battery

The integration of PINN and DON, called multiphysics-informed deep operator network (MPI-DON or MPI-DeepONet) incorporating encoders, is proposed for predicting thermal runaway (TR) under a range of heating conditions, providing a rapid and accurate surrogate model for TR prediction. This neural network uses the

heating curve, representing thermal abuse conditions, as input to output the chemical concentrations of the positive electrode, negative electrode, solid electrolyte interphase (SEI), and electrolyte along with the internal temperature response of the lithium-ion battery (LIB).

MPI-DON incorporating encoders have three main features that play a crucial role in attaining accurate final stage TR predictions, organized into three phases (**Figure 9(a)**). First, phase 1 generates virtual data by employing FEA [64] at various heating conditions (**Figure 9(a)**, red box). Specifically, virtual data is generated to reflect the mechanism of chemical decomposition of LIB components due to temperature-dependent chain reactions involving positive electrode, negative electrode, SEI, and electrolyte. This mechanism is governed by the energy balance equation [65], Arrhenius equation [66], and heat transfer equations. MPI-DON incorporating encoders are trained with virtual data because it is challenging and expensive to conduct experiments for all operating conditions. This strategy also facilitates MPI-DON incorporating encoders in capturing hidden physics, such as four main chemical concentrations and the internal temperature response of the LIB during the training phase (**Figure 9(a)**, purple box). Therefore, the proposed neural network can function as a virtual sensor in practical uses by acquiring this key information through FEA. Note that virtual data generated under various heating conditions are separated into a training set and a test set, with the test set validating the capability of interpolation and extrapolation. Second, DON serves as a surrogate model for predicting TR in untrained conditions, utilizing the branch network and the trunk network for operator regression (**Figure 9(b)**, blue dashed box). As explained in Subsection 3.3, this strategy uses the branch and trunk networks to map one function (i.e., the heating condition) to another function (i.e., the responses of the cell) to approximate the solution of a nonlinear system of equations, which outperforms traditional neural networks that have to be retrained for every unseen condition. Furthermore, encoders [55] are added to the architecture of DON to resolve the issue of restricted information exchange, where branch network and trunk network interact only in the last layer. These encoders improve the ability of the DON to capture the complex response of the TR phenomenon by smoothing the input information flow to each hidden layer. Third, incorporating a novel training strategy that supervises multiple equations governing the TR of LIBs (**Figure 9(b)**, green dashed box), along with adaptive weights allocation based on the NTK method (**Figure 9(b)**, pink dashed box), can enhance the robustness and accuracy of the DON with encoders during the training phase. Thus, MPI-DON incorporating encoders can achieve convergence to the global minimum, thanks to the governing equations regulating the training process. Note that the input values and output values are normalized to convert the dimensionless values, addressing the mismatch between the neural network values and the physical quantities represented by the governing equations. After completing phase 2, phase 3 predicts the hidden states of a LIB, encompassing chemical concentrations and internal temperature distribution at diverse heating conditions (**Figure 9(a)**, green box).

The effectiveness of physics constraints is demonstrated by comparing MPI-DON incorporating encoders supervised by physics constraints to DON that relies solely on data during training. **Figure 10** shows an example of predicting an interpolation task in a test set, confirming the effectiveness of physics constraints for improving accuracy. Specifically, MPI-DON, achieving an RMSE of 7.6°C in temperature predictions (**Figure 10(a)**, red line), accurately predicts temperature evolutions over all time domains, even in regions without labeled data points. In contrast, DON, achieving an RMSE of 30.5°C in temperature predictions (**Figure 10(a)**, blue line), shows large

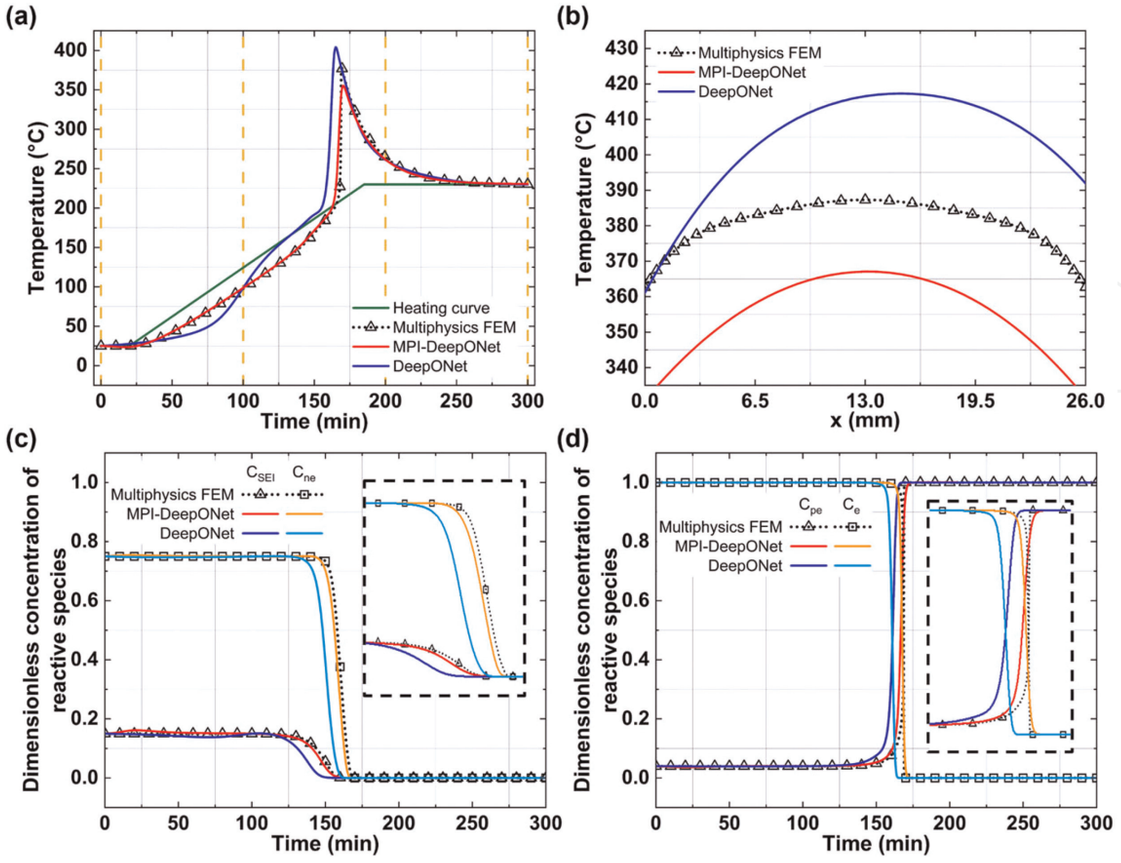


Figure 10.
Comparison of MPI-DON and DON with multiphysics FEMs for TR in test set 1: (a) predicted mean temperature evolution, (b) internal LIB temperature distribution at TR peak, (c) predicted mean c_{SEI} and c_{ne} evolution, and (d) predicted mean c_{pe} and c_e evolution [37].

errors in the time domain without labeled data points when TR occurs. Remarkably, DON shows little error on the trained data points labeled 0, 100, 200, and 300 min (**Figure 10(a)**, yellow dashed line), but significant error on the untrained data points. MPI-DON predicts responses accurately across all time domains due to physics constraints. This result suggests that purely data-driven methods may be incapable of predicting the multiphysics response of LIBs, resulting in convergence to a local minimum due to insufficient training data. **Figure 10(b)** compares the internal temperature distribution along the center cross-section line of the LIB at the peak point, illustrating results from multiphysics FEM (black dotted line with triangles), MPI-DON (red line), and DON (blue line). This result reveals the role of physics constraints in ensuring predictions match the multiphysics characteristics of LIBs. Specifically, the symmetrical internal temperature distributions within the LIB shown by multiphysics FEM and MPI-DON, centered around the midpoint due to isotropic conduction, contrast with the asymmetric temperature distribution produced by DON, confirming the effectiveness of physics constraints. In **Figure 10(c)** and **(d)**, the chemical concentrations are compared among multiphysics FEM (black dotted lines with triangles and squares), MPI-DON (red and orange lines), and DON (blue and light blue lines). The inset figure shows that MPI-DON accurately predicts the sharp variations in chemical concentration during TR, in contrast to DON, which shows significant inaccuracies. This result confirms that MPI-DON accurately predicts TR occurrence from heating conditions, predicting key chemical concentrations and temperature accurately.

The contribution of supervision from multiphysics is highlighted by comparing predictions from MPI-DON and DON based on varying numbers of data collocation points. Data collocation intervals of 15, 20, 30, 50, 60, and 100 minutes correspond to 14,364, 10,944, 7524, 4788, 4104, and 2736 points, respectively, for training the neural networks. All other training conditions, including physics, initial, and boundary condition collocation points, remain constant. The mean and 95% confidence interval of RMSE for temperature prediction are determined for the test sets with five random initial parameters (**Figure 11**). Results show that MPI-DON consistently predicts TR phenomena more accurately than DON, regardless of data collocation point density, due to supervision by multiphysics characteristics. Conversely, RMSE of DON increases significantly as data become sparse, indicating a high dependence on data. The confidence interval for MPI-DON is narrower, especially with 100-minute intervals, indicating better stability even with sparse data. This observation suggests robustness of MPI-DON in industrial applications where data is often limited. While DON is cost-effective with ample data (intervals less than 30 minutes), physics constraints are generally advantageous for real-world applications.

MPI-DON incorporating encoders offers a significantly faster inference time compared to multiphysics FEM. Specifically, MPI-DON takes just 0.5 ms for TR prediction, while multiphysics FEM takes about 7 seconds, making MPI-DON 10,000 times faster. After a 720-minute training phase, MPI-DON continues to provide instant predictions, unlike multiphysics FEM, whose prediction time increases linearly with the number of heating cases. This efficiency makes MPI-DON more versatile and effective for testing LIB reliability under various conditions through Monte Carlo analysis, a task that would be time-consuming and costly with FEM. The speed of MPI-DON also suits real-time applications, such as TR management in thermal systems.

In conclusion, this approach overcomes the limitations of traditional numerical methods and data-driven techniques, offering a solution that is both computationally

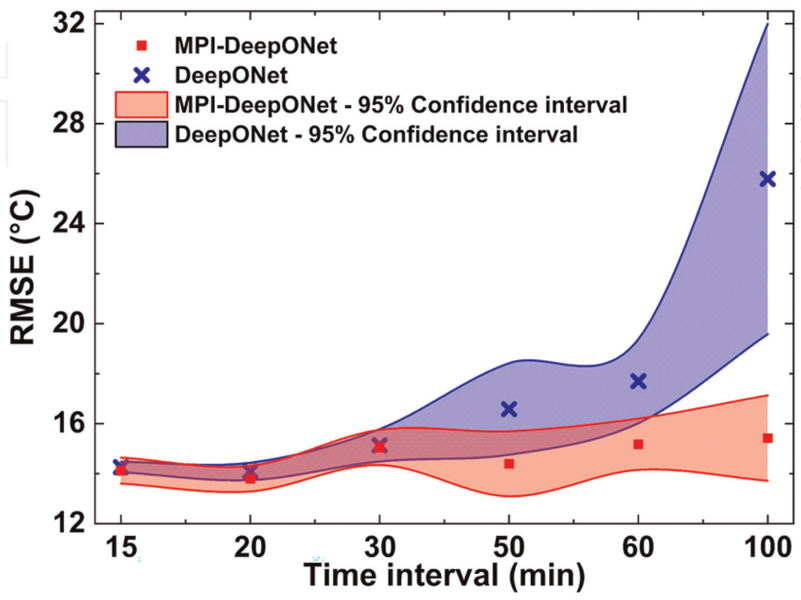


Figure 11.
Temperature RMSE of MPI-DON and DON across various time interval of the data collocation points [37].

efficient and highly accurate. The proposed neural network offers indispensable data for sophisticated thermal battery management systems, thereby improving the reliability and safety of LIBs. This advancement propels the development of digital twins for electric transport systems through artificial intelligence transformation. Additionally, beyond addressing the TR phenomenon, this approach can be implemented in various systems, furthering the impact of artificial intelligence transformation across various domains.

5. Conclusion

This chapter delved into the potential of artificial intelligence, particularly focusing on PINN, which are adept at handling complex and nonlinear systems in real-world engineering. PINN effectively addresses challenges such as data scarcity and imbalance by integrating physics and fundamental principles during the training phase. PINN ensures predictions adhere to physical laws, even when data is sparse, by embedding governing equations as constraints within the loss function. However, incorporating physical constraints into neural networks increases the training complexity, necessitating appropriate strategies for optimal performance. Especially, strategies to address complexity become crucial when applying PINN to multiphysics systems, where multiple physics are coupled. Therefore, this chapter also introduced various strategies for applying PINN to mechatronic systems such as motor and battery. The principles of PINN summarized and the introduced framework for applying PINN to multiphysics systems will be crucial in advancing technological innovation, ensuring system reliability, and influencing the future of engineering and applied sciences.

Acknowledgements

This work was supported by the Air Force Office of Scientific Research under award number FA2386-23-1-4094, by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) [grant number 2020R1C1C1003829], by the Korea Institute of Machinery and Materials [grant No. NK250B].

Nomenclature

DNN	deep neural network
DeepONet	deep operator network
FEM	finite element model
LIB	lithium-ion battery
MLP	multi-layer perceptron
MPI-DeepONet	multiphysics-informed deep operator network
MSE	mean squared error
NTK	neural tangent kernel
PDE	partial differential equation
PHM	prognostics and health management
PINN	physics informed neural network

RMSE	root mean squared error
SEI	solid electrolyte interphase
TR	thermal runaway

Symbols

$\boldsymbol{B}$	magnetic flux density
$\boldsymbol{b}_*^{(l)}$	bias of the lth hidden layer
c_e	value of electrolyte
c_{ne}	value of negative electrode
c_{pe}	value of positive electrode
c_{SEI}	value of SEI
$\boldsymbol{f}$	final output of the neural network
$\mathbf{F^i}$	Fourier feature mapping matrix for the i-th embedding
G_*	operator output
$\boldsymbol{H}_*^{(l)}$	output of the lth hidden layer
H^{NTK}	NTK matrix
$\boldsymbol{I}$	indicator function
L	total loss function
N_*	number of collocation points
$\boldsymbol{O}_*$	final output
$\boldsymbol{\Phi}$	Fourier feature mapping of input
Res_*	residual of governing equation
T	time span
L_*	loss term
$\boldsymbol{W}$	weight for each loss function
$\boldsymbol{W}_*^{(l)}$	weight of the lth hidden layer

Greek letters

λ_*	adaptive weights
θ	trainable parameters
u	reference solution of the PDE

Subscripts

0	initial value
BC	boundary condition
D	data fitting
I	initial condition
P	physics
q	subdomain
u	branch network
y	trunk network

Superscripts

$\hat{}$	predicted solution of neural network

Physics-Informed Neural Network: Principles and Applications
DOI: http://dx.doi.org/10.5772/intechopen.1006130

IntechOpen

Author details

Seho Son[1], Jinho Jeong[1], Dayeon Jeong[1], Kyung ho Sun[2] and Ki-Yong Oh[1,3]*

1 Department of Mechanical Convergence Engineering, Hanyang University, Seoul, Republic of Korea

2 Virtual Engineering Research Center, Korea Institute of Machinery and Materials, Daejeon, Republic of Korea

3 School of Mechanical Engineering, Hanyang University, Seoul, Republic of Korea

*Address all correspondence to: kiyongoh@hanyang.ac.kr

IntechOpen

© 2024 The Author(s). Licensee IntechOpen. This chapter is distributed under the terms of the Creative Commons Attribution License (http://creativecommons.org/licenses/by/4.0), which permits unrestricted use, distribution, and reproduction in any medium, provided the original work is properly cited. CC BY

References

[1] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez AN, et al. Attention is all you need. Advances in Neural Information Processing Systems. 2017;**30**. DOI: 10.48550/arXiv.1706.03762

[2] Devlin J, Chang M-W, Lee K, Toutanova K. Bert: Pre-Training of Deep Bidirectional Transformers for Language Understanding. arXiv preprint arXiv: 1810.04805. 2018. DOI: 10.48550/arXiv.1810.04805

[3] Lei S, Yi W, Ying C, Ruibin W. Review of attention mechanism in natural language processing. Data Analysis and Knowledge Discovery. 2020;**4**:1-14. DOI: 10.11925/infotech.2096-3467.2019.1317

[4] Krizhevsky A, Sutskever I, Hinton GE. Imagenet classification with deep convolutional neural networks. Communications of the ACM. 2012;**60**:84-90. DOI: 10.1145/3065386

[5] Redmon J, Divvala S, Girshick R, Farhadi A. You only look once: Unified, real-time object detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 779-788

[6] Yin Y, Li H, Fu W. Faster-YOLO: An accurate and faster object detection method. Digital Signal Processing. 2020; **102**:102756. DOI: 10.1016/j.dsp.2020.102756

[7] Kim D, Kim S, Jeong S, Ham JW, Son S, Moon J, et al. Rotational multipyramid network with bounding-box transformation for object detection. International Journal of Intelligent Systems. 2021;**36**:5307-5338. DOI: 10.1002/int.22513

[8] Moon J, Jeon M, Jeong S, RoMP-transformer K-YO. Rotational bounding box with multi-level feature pyramid transformer for object detection. Pattern Recognition. 2024;**147**:110067. DOI: 10.1016/j.patcog.2023.110067

[9] Zhang S, Zhang S, Wang B, Habetler TG. Deep learning algorithms for bearing fault diagnostics—A comprehensive review. IEEE Access. 2020;**8**:29857-29881. DOI: 10.1109/ACCESS.2020.2972859

[10] Çınar ZM, Abdussalam Nuhu A, Zeeshan Q, Korhan O, Asmael M, Safaei B. Machine learning in predictive maintenance towards sustainable smart manufacturing in industry 4.0. Sustainability. 2020;**12**:8211. DOI: 10.3390/su12198211

[11] Jeong Y. Digitalization in production logistics: How AI, digital twins, and simulation are driving the shift from model-based to data-driven approaches. International Journal of Precision Engineering and Manufacturing-Smart Technology. 2023;**1**:187-200. DOI: 10.57062/ijpem-st.2023.0052

[12] Kim M, Son S, Oh K-Y. Margin-maximized hyperspace for fault detection and prediction: A case study with an elevator door. IEEE Access. 2023;**11**:128580-128595. DOI: 10.1109/ACCESS.2023.3330137

[13] Son S, Oh K-Y. Integrated framework for estimating remaining useful lifetime through a deep neural network. Applied Soft Computing. 2022; **122**:108879. DOI: 10.1016/j.asoc.2022.108879

[14] Hornik K, Stinchcombe M, White H. Multilayer feedforward networks are universal approximators. Neural

Networks. 1989;**2**:359-366. DOI: 10.1016/0893-6080(89)90020-8

[15] Huang Z, Shen Y, Li J, Fey M, Brecher C. A survey on AI-driven digital twins in industry 4.0: Smart manufacturing and advanced robotics. Sensors. 2021;**21**:6340. DOI: 10.3390/s21196340

[16] Wang J, Li Y, Gao RX, Zhang F. Hybrid physics-based and data-driven models for smart manufacturing: Modelling, simulation, and explainability. Journal of Manufacturing Systems. 2022;**63**:381-391. DOI: 10.1016/j.jmsy.2022.04.004

[17] Rai R, Sahu CK. Driven by data or derived through physics? A review of hybrid physics guided machine learning techniques with cyber-physical system (cps) focus. IEEE Access. 2020;**8**:71050-71073. DOI: 10.1109/ACCESS.2020.2987324

[18] Chen H, Lou S, Lv C. Hybrid physics-data-driven online modelling: Framework, methodology and application to electric vehicles. Mechanical Systems and Signal Processing. 2023;**185**:109791. DOI: 10.1016/j.ymssp.2022.109791

[19] Son S, Jeong S, Kwak E, Kim J-H, Oh K-Y. Integrated framework for SOH estimation of lithium-ion batteries using multiphysics features. Energy. 2022;**238**:121712. DOI: 10.1016/j.energy.2021.121712

[20] Kumar K, Choi Y. Accelerating particle and fluid simulations with differentiable graph networks for solving forward and inverse problems. Proceedings of the SC'23 Workshops of the International Conference on High Performance Computing, Network, Storage, and Analysis. 2023. pp. 60-65

[21] Raissi M, Perdikaris P, Karniadakis GE. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. Journal of Computational Physics. 2019;**378**:686-707. DOI: 10.1016/j.jcp.2018.10.045

[22] Tartakovsky AM, Marrero CO, Perdikaris P, Tartakovsky GD, Barajas-Solano D. Physics-informed deep neural networks for learning parameters and constitutive relationships in subsurface flow problems. Water Resources Research. 2020;**56**:e2019WR026731. DOI: 10.1029/2019WR026731

[23] Cao Y, Fang Z, Wu Y, Zhou D-X, Gu Q. Towards Understanding the Spectral Bias of Deep Learning. arXiv preprint arXiv:1912.01198. 2019. DOI: 10.48550/arXiv.1912.01198

[24] Wang S, Sankaran S, Perdikaris P. Respecting Causality is All You Need for Training Physics-Informed Neural Networks. arXiv preprint arXiv:2203.07404. 2022. DOI: 10.48550/arXiv.2203.07404

[25] Bischof R, Kraus M. Multi-Objective Loss Balancing for Physics-Informed Deep Learning. arXiv preprint arXiv:2110.09813. 2021. DOI: 10.48550/arXiv.2110.09813

[26] McClenny LD, Braga-Neto UM. Self-adaptive physics-informed neural networks. Journal of Computational Physics. 2023;**474**:111722. DOI: 10.1016/j.jcp.2022.111722

[27] Krishnapriyan A, Gholami A, Zhe S, Kirby R, Mahoney MW. Characterizing possible failure modes in physics-informed neural networks. Advances in Neural Information Processing Systems. 2021;**34**:26548-26560. DOI: 10.48550/arXiv.2109.01050

[28] Wang S, Teng Y, Perdikaris P. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. SIAM Journal on Scientific Computing. 2021;**43**:A3055-A3081. DOI: 10.1137/20M1318043

[29] Jagtap AD, Kharazmi E, Karniadakis GE. Conservative physics-informed neural networks on discrete domains for conservation laws: Applications to forward and inverse problems. Computer Methods in Applied Mechanics and Engineering. 2020;**365**: 113028. DOI: 10.1016/j.cma.2020.113028

[30] Kharazmi E, Zhang Z, Karniadakis GE. hp-VPINNs: Variational physics-informed neural networks with domain decomposition. Computer Methods in Applied Mechanics and Engineering. 2021;**374**:113547. DOI: 10.1016/j.cma.2020.113547

[31] Lu L, Jin P, Pang G, Zhang Z, Karniadakis GE. Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. Nature Machine Intelligence. 2021;**3**:218-229. DOI: 10.1038/s42256-021-00302-5

[32] Li Z, Kovachki N, Azizzadenesheli K, Liu B, Bhattacharya K, Stuart A, et al. Fourier Neural Operator for Parametric Partial Differential Equations. arXiv preprint arXiv:2010.08895. 2020. DOI: 10.48550/arXiv.2010.08895

[33] Tancik M, Srinivasan P, Mildenhall B, Fridovich-Keil S, Raghavan N, Singhal U, et al. Fourier features let networks learn high frequency functions in low dimensional domains. Advances in Neural Information Processing Systems. 2020; **33**:7537-7547. DOI: 10.48550/arXiv.2006.10739

[34] Basri R, Galun M, Geifman A, Jacobs D, Kasten Y, Kritchman S. Frequency bias in neural networks for input of non-uniform density. International Conference on Machine Learning. PMLR; 2020. pp. 685-694

[35] Niaki SA, Haghighat E, Campbell T, Poursartip A, Vaziri R. Physics-informed neural network for modelling the thermochemical curing process of composite-tool systems during manufacture. Computer Methods in Applied Mechanics and Engineering. 2021;**384**:113959. DOI: 10.1016/j.cma.2021.113959

[36] Cai S, Wang Z, Wang S, Perdikaris P, Karniadakis GE. Physics-informed neural networks for heat transfer problems. Journal of Heat Transfer. 2021;**143**:060801. DOI: 10.1115/1.4050542

[37] Jeong J, Kwak E, Kim J-H, Oh K-Y. Prediction of thermal runaway for a lithium-ion battery through multiphysics-informed DeepONet with virtual data. eTransportation. 2024;**21**: 100337. DOI: 10.1016/j.etran.2024.100337

[38] Mao Z, Jagtap AD, Karniadakis GE. Physics-informed neural networks for high-speed flows. Computer Methods in Applied Mechanics and Engineering. 2020;**360**:112789. DOI: 10.1016/j.cma.2019.112789

[39] Cai S, Mao Z, Wang Z, Yin M, Karniadakis GE. Physics-informed neural networks (PINNs) for fluid mechanics: A review. Acta Mechanica Sinica. 2021;**37**:1727-1738. DOI: 10.1007/s10409-021-01148-1

[40] Son S, Lee H, Jeong D, Oh K-Y, Sun KH. A novel physics-informed neural network for modeling electromagnetism of a permanent

magnet synchronous motor. Advanced Engineering Informatics. 2023;**57**: 102035. DOI: 10.1016/j.aei.2023.102035

[41] Kollmannsberger S, D'Angella D, Jokeit M, Herrmann L. Deep Learning in Computational Mechanics. Springer International Publishing; 2021. DOI: 10.1007/978-3-030-76587-3

[42] Diao Y, Yang J, Zhang Y, Zhang D, Du Y. Solving multi-material problems in solid mechanics using physics-informed neural networks based on domain decomposition technology. Computer Methods in Applied Mechanics and Engineering. 2023;**413**: 116120. DOI: 10.1016/j.cma.2023.116120

[43] Faroughi SA, Pawar NM, Fernandes C, Raissi M, Das S, Kalantari NK, et al. Physics-guided, physics-informed, and physics-encoded neural networks and operators in scientific computing: Fluid and solid mechanics. Journal of Computing and Information Science in Engineering. 2024;**24**:040802. DOI: 10.1115/1.4064449

[44] Zhang E, Dao M, Karniadakis GE, Suresh S. Analyses of internal structures and defects in materials using physics-informed neural networks. Science Advances. 2022;**8**:eabk0644. DOI: 10.1126/sciadv.abk0644

[45] Lou Q, Meng X, Karniadakis GE. Physics-informed neural networks for solving forward and inverse flow problems via the Boltzmann-BGK formulation. Journal of Computational Physics. 2021;**447**:110676. DOI: 10.1016/j.jcp.2021.110676

[46] Jagtap AD, Karniadakis GE. Extended physics-informed neural networks (XPINNs): A generalized space-time domain decomposition based deep learning framework for nonlinear partial differential equations. Communications in Computational Physics. 2020;**28**:5. DOI: 10.4208/cicp.oa-2020-0164

[47] Raissi M. Forward–backward stochastic neural networks: Deep learning of high-dimensional partial differential equations. Peter Carr Gedenkschrift: Research Advances in Mathematical Finance. World Scientific; 2024. pp. 637-655

[48] Sirignano J, Spiliopoulos K. DGM: A deep learning algorithm for solving partial differential equations. Journal of Computational Physics. 2018;**375**: 1339-1364. DOI: 10.1016/j.jcp.2018.08.029

[49] Shukla K, Jagtap AD, Karniadakis GE. Parallel physics-informed neural networks via domain decomposition. Journal of Computational Physics. 2021;**447**: 110683. DOI: 10.1016/j.jcp.2021.110683

[50] Wang S, Yu X, Perdikaris P. When and why PINNs fail to train: A neural tangent kernel perspective. Journal of Computational Physics. 2022;**449**: 110768. DOI: 10.1016/j.jcp.2021.110768

[51] Guan H, Dong J, Lee W-N. Towards Real-time Training of Physics-informed Neural Networks: Applications in Ultrafast Ultrasound Blood Flow Imaging. arXiv preprint arXiv: 2309.04755. 2023. DOI: 10.48550/arXiv.2309.04755

[52] He J, Koric S, Kushwaha S, Park J, Abueidda D, Jasiuk I. Novel DeepONet architecture to predict stresses in elastoplastic structures with variable complex geometries and loads. Computer Methods in Applied Mechanics and Engineering. 2023; **415**:116277. DOI: 10.1016/j.cma.2023.116277

[53] Koric S, Abueidda DW. Data-driven and physics-informed deep learning operators for solution of heat conduction equation with parametric heat source. International Journal of Heat and Mass Transfer. 2023;**203**:123809. DOI: 10.1016/j.ijheatmasstransfer.2022.123809

[54] Goswami S, Yin M, Yu Y, Karniadakis GE. A physics-informed variational DeepONet for predicting crack path in quasi-brittle materials. Computer Methods in Applied Mechanics and Engineering. 2022;**391**: 114587. DOI: 10.1016/j.cma.2022.114587

[55] Wang S, Wang H, Perdikaris P. Improved architectures and training algorithms for deep operator networks. Journal of Scientific Computing. 2022;**92**:35. DOI: 10.1007/s10915-022-01881-0

[56] He J, Kushwaha S, Park J, Koric S, Abueidda D, Jasiuk I. Sequential deep operator networks (S-DeepONet) for predicting full-field solutions under time-dependent loads. Engineering Applications of Artificial Intelligence. 2024;**127**:107258. DOI: 10.1016/j.engappai.2023.107258

[57] Rahaman N, Baratin A, Arpit D, Draxler F, Lin M, Hamprecht F, et al. On the spectral bias of neural networks. International Conference on Machine Learning. PMLR; 2019. pp. 5301-5310

[58] Wang S, Wang H, Perdikaris P. On the eigenvector bias of Fourier feature networks: From regression to solving multi-scale PDEs with physics-informed neural networks. Computer Methods in Applied Mechanics and Engineering. 2021;**384**:113938. DOI: 10.1016/j.cma.2021.113938

[59] Hesthaven JS, Gottlieb S, Gottlieb D. Spectral Methods for Time-Dependent Problems. Cambridge University Press; 2007;**21**. DOI: 10.1017/CBO9780511618352

[60] Jabbar M, Liu Z, Dong J. Time-stepping finite-element analysis for the dynamic performance of a permanent magnet synchronous motor. IEEE Transactions on Magnetics. 2003;**39**: 2621-2623. DOI: 10.1109/TMAG.2003.816500

[61] Gherabi Z, Toumi D, Benouzza N, Boudinar AH, Koura MB. Discrimination between demagnetization and eccentricity faults in PMSMs using real and imaginary components of stator current spectral analysis. Journal of Power Electronics. 2021;**21**:153-163. DOI: 10.1007/s43236-020-00169-6

[62] Hyunseung Lee SS, Jeong D, Sun KH, Jeon BC, Ki-Yong O. High-fidelity multiphysics model of a permanent magnet synchronous motor for fault data generation. Journal of Sound and Vibration. 2024;**589**:118573. DOI:10.1016/j.jsv.2024.118573

[63] Lee H, Son S, Jeong D, Sun KH, Jeon BC, Oh K-Y. A Finite Element Model of an Electric Motor with an Unbalanced Rotor for Vibration Data Generation. International Journal of Precision Engineering and Manufacturing-Smart Technology. 2024;**2**(1):47-56. DOI: 10.57062/ijpem-st.2023.0122

[64] Kwak E, Kim J-H, Jeong J, Oh K-Y. Multiphysics-informed thermal runaway model for estimating the pressure evolution induced by the gas formation in a lithium-ion battery. Applied Thermal Engineering. 2024;**246**:122941. DOI: 10.1016/j.applthermaleng.2024.122941

[65] Guo G, Long B, Cheng B, Zhou S, Xu P, Cao B. Three-dimensional thermal finite element modeling of lithium-ion

battery in thermal abuse application. Journal of Power Sources. 2010;**195**: 2393-2398. DOI: 10.1016/j.jpowsour.2009.10.090

[66] Liu B, Jia Y, Yuan C, Wang L, Gao X, Yin S, et al. Safety issues and mechanisms of lithium-ion battery cell upon mechanical abusive loading: A review. Energy Storage Materials. 2020; **24**:85-112. DOI: 10.1016/j.ensm.2019.06.036

IntechOpen